

Documento de Arquitectura de Solución

1. Introducción

1.1 Propósito

Este documento describe la arquitectura de la solución para el **Sistema de Banca por Internet BP**, incluyendo sus funcionalidades, componentes internos, integraciones y las decisiones técnicas fundamentadas para garantizar alta disponibilidad y seguridad.

1.2 Alcance

La solución abarca la creación de **dos aplicaciones front-end (una SPA web y una App Móvil nativa)**, y un **back-end basado en microservicios alojados en la nube de AWS**. Incluye la implementación de un API Gateway, seguridad centralizada mediante OAuth 2.0, bases de datos propias (Caché y Auditoría) e integraciones con sistemas biométricos y notificaciones. No incluye la modernización ni modificación interna de la Plataforma Core heredada del banco.

1.3 Definiciones y Acrónimos

- **SPA:** Single Page Application (Aplicación web de una sola página).
- **OAuth 2.0:** Estándar de autorización de seguridad.
- **AWS:** Amazon Web Services (Proveedor Cloud).
- **Saga:** Patrón de arquitectura para orquestar transacciones distribuidas.
- **Circuit Breaker:** Patrón para proteger sistemas ante fallos de latencia o caídas.
- **Cache-Aside:** Estrategia para almacenar datos temporales en memoria ultrarrápida.

2. Descripción General del Sistema

2.1 Contexto del Negocio

- **Problema que se busca resolver:** La entidad bancaria **BP** requiere el diseño de un nuevo sistema de banca por internet que permita a sus usuarios gestionar su salud

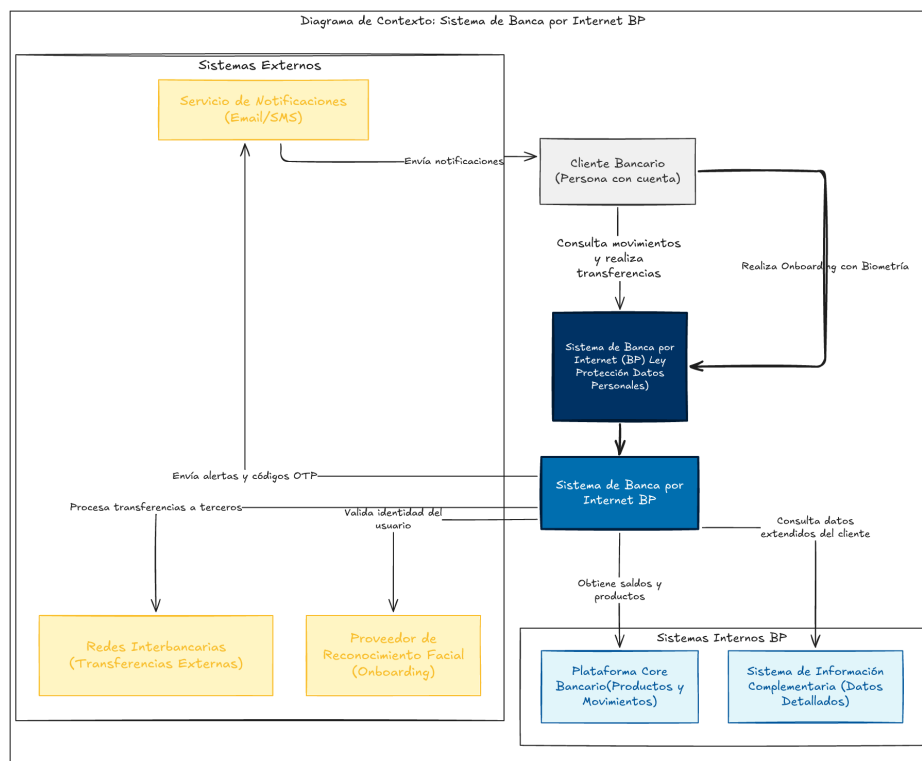
financiera de forma integral y segura. El desafío principal radica en consolidar la información proveniente de dos fuentes distintas: una **Plataforma Core** (datos básicos y productos) y un **Sistema de Información Complementaria** (detalles extendidos), garantizando al mismo tiempo el cumplimiento de las normativas vigentes sobre notificaciones al usuario.

- **Stakeholders:** Clientes de BP, Auditores de seguridad, Administradores del Banco.
- **Objetivos del negocio:** Proveer una experiencia bancaria rápida y accesible 24/7, asegurar el cumplimiento de normativas financieras (ley de datos y notificaciones), y aislar la nueva plataforma de las caídas de los sistemas *legacy*.

2.2 Actores del Sistema

- **Usuario final:** Cliente Bancario (persona con cuenta).
- **Sistemas externos:** Proveedor de Reconocimiento Facial, Redes Interbancarias, y Servicio de Notificaciones (Email/SMS).
- **Sistemas internos (BP):** Plataforma Core Bancario y Sistema de Información Complementaria.

2.3 Diagrama de Contexto (C4 - Nivel 1)



3. Funcionalidades del Sistema

3.1 Capacidades Principales

- **Gestión de usuarios:** Onboarding digital con biometría facial y autenticación segura.
- **Procesamiento de transacciones:** Transferencias propias, interbancarias y pagos.
- **Notificaciones:** Alertas automáticas por SMS y Push para cumplir la regulación.
- **Gestión Integral de cuentas:** Consulta ágil del historial de movimientos y saldos mediante el uso de cachés.

3.2 Descripción de Funcionalidades

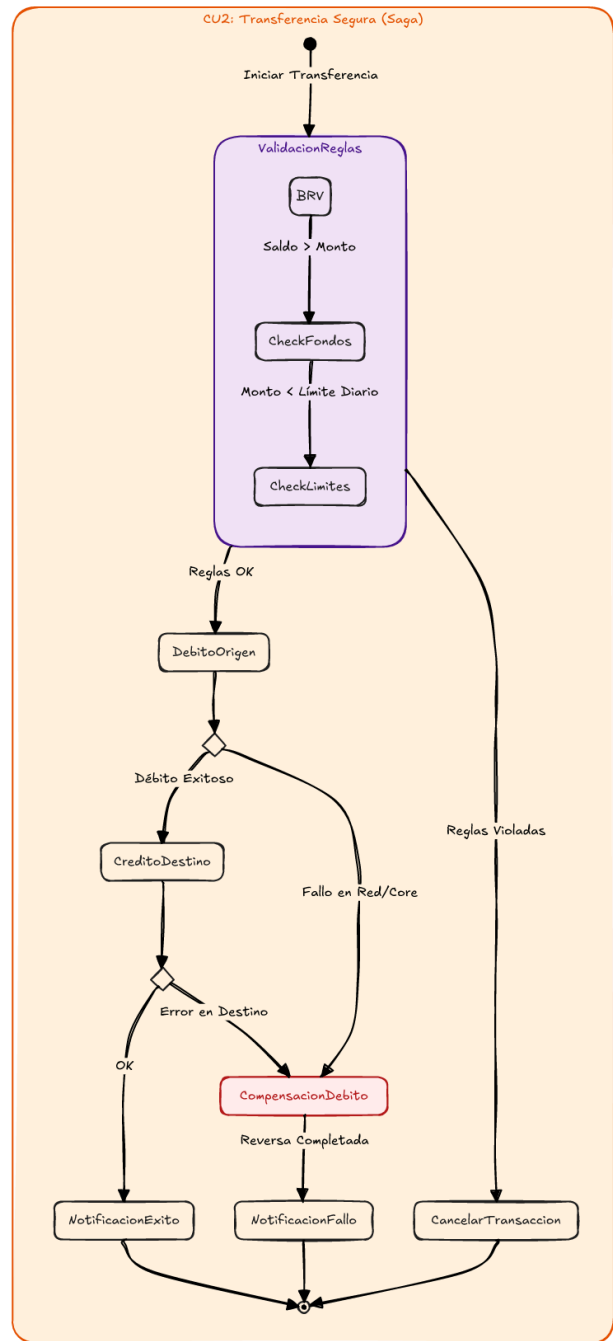
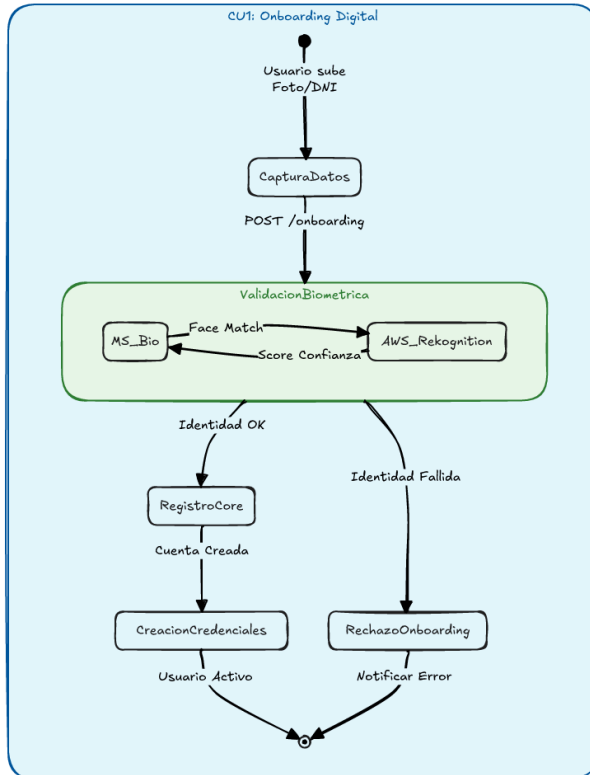
Funcionalidad	Descripción	Prioridad
F1: Onboarding y Biometría	Registro de clientes nuevos verificando identidad con reconocimiento facial (AWS Rekognition).	Alta
F2: Transferencias y Pagos	Movimiento de fondos aplicando reglas de negocio estrictas y transacciones orquestadas.	Alta
F3: Consulta de Saldos	Vista de datos de clientes y saldos de forma casi instantánea usando memoria rápida.	Alta
F4: Auditoría Inmutable	Registro automático de acciones críticas en una base de datos exclusiva que no puede ser alterada.	Media

4. Casos de Uso y Flujos

4.1 Casos de Uso Principales

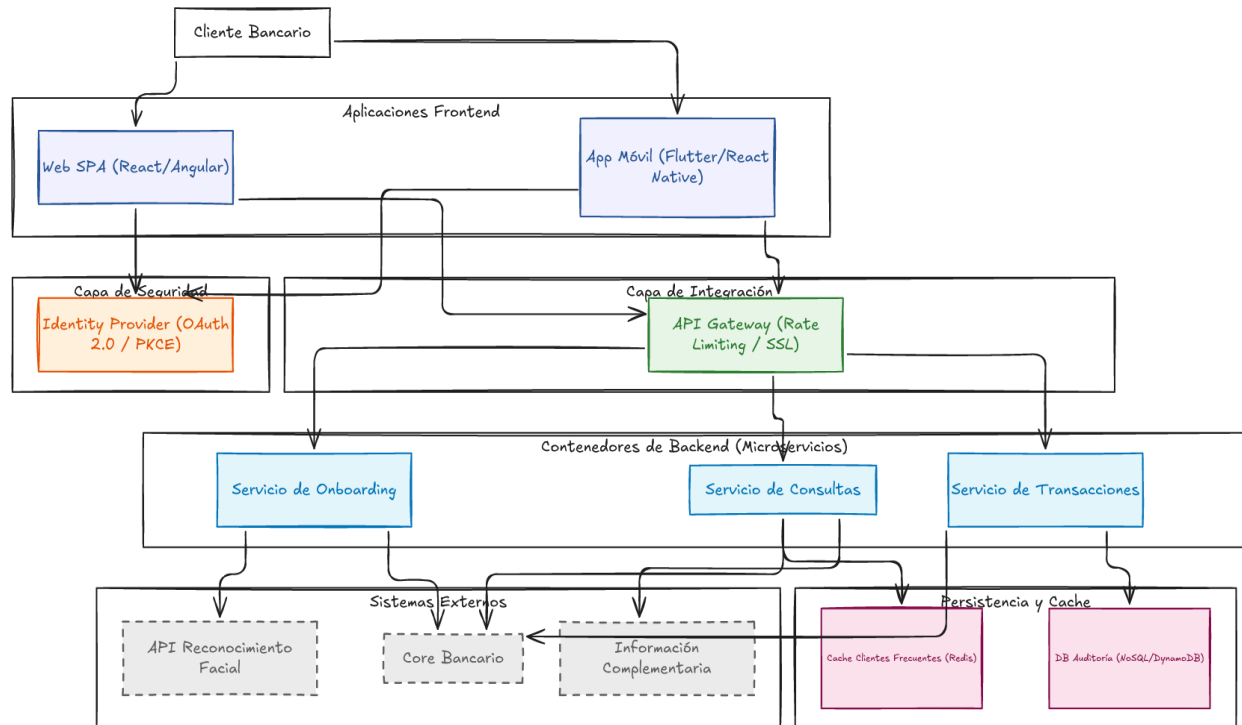
- **CU1: Onboarding Digital.** El usuario toma una foto y el microservicio de Onboarding valida contra el API de Reconocimiento Facial para permitirle crear credenciales (huella/clave).
- **CU2: Transferencia Segura.** El usuario inicia la transferencia, el "Business Rules Validator" verifica fondos y límites; el "Saga Orchestrator" asegura que la transacción complete todos sus pasos o se revierta en caso de error en el Core Bancario.

4.2 Diagramas de Secuencia / Flujos



5. Arquitectura de la Solución

5.1 Vista de Contenedores (C4 - Nivel 2)



5.2 Descripción de Componentes

Componente	Descripción	Responsabilidad
Identity Provider	AWS Cognito / OAuth2 Provider.	Gestiona el flujo OAuth 2.0 para accesos seguros.
API Gateway	AWS API Gateway.	Único punto de entrada; maneja routing, seguridad y rate limiting.
Microservicio Onboarding	AWS Lambda / ECS.	Registro e interacción con el motor biométrico.
Microservicio Consultas	Contenedor de backend.	Busca datos de usuario priorizando la caché antes que el Core.

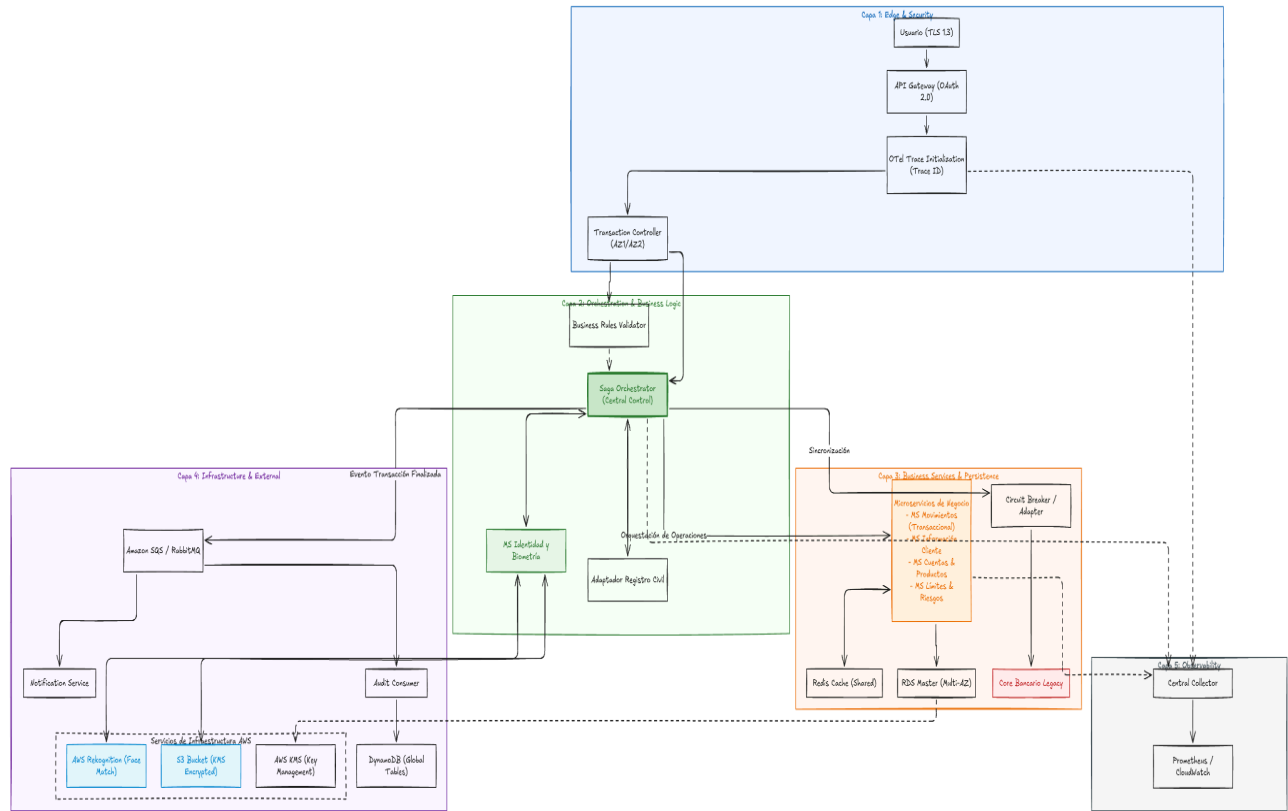
Componente	Descripción	Responsabilidad
Microservicio Transacciones	Contenedor de backend.	Orquesta lógicas complejas de transferencias y pagos.

6. Diseño de Componentes (C4 - Nivel 3)

6.1 Detalle por Servicio

6.1.1 Microservicio de Transferencias

- **Controladores:** **Transaction Controller** expone endpoints REST y recibe la solicitud validada por el API Gateway.
- **Lógica de negocio:** Un **Business Rules Validator** actúa como primer filtro lógico. El núcleo es el **Saga Orchestrator**, el cual asegura que el pago se haga completo de manera distribuida.
- **Acceso a datos / Integración:** Utiliza una **Adapter Layer** (Capa Anticorrupción) para traducir datos modernos al lenguaje del banco antiguo, protegida por un **Circuit Breaker** que evita que el sistema caiga si el Core responde lento.



7. Modelo de Datos

7.1 Entidades Principales

- **Usuario:** Contiene perfiles y vínculos a registros biométricos cifrados.
- **Transacción:** Historial de transferencias y estados manejados por el orquestador Saga.
- **Auditoría:** Registro de eventos (Trace ID) inmutables guardados en formato NoSQL.

8. Integraciones

8.1 Sistemas Externos

Sistema	Tipo de integración	Descripción
API Reconocimiento Facial	REST	Se utiliza AWS Rekognition para validar biometría en el onboarding.
Core Bancario Legacy	mTLS / Adaptador	Sistema antiguo del banco para transacciones, protegido vía Circuit Breaker.

8.2 Comunicación Interna

- **REST:** Comunicación síncrona en los controladores.
- **Mensajería / Eventos (SQS/RabbitMQ):** Uso asíncrono implementado en el "Notification Publisher" para despachar SMS/Emails sin detener el proceso principal del usuario.

9. Atributos de Calidad

9.1 Escalabilidad

El sistema escala horizontalmente al desplegar microservicios independientes sobre la nube de AWS (Amazon ECS/Lambdas). Cada servicio del banco puede crecer a su propio ritmo.

9.2 Disponibilidad

Se aplica alta disponibilidad (HA) desplegando los controladores en **Múltiples Zonas de Disponibilidad (Multi-AZ)**. La base de datos principal (RDS) opera en modelo Master Multi-AZ. Como recuperación ante desastres (DR), se automatizan respaldos geográficos mediante "Cross-Region Snapshots".

9.3 Seguridad

- **Autenticación:** Flujo **OAuth 2.0 (Authorization Code)**; previene interceptación de credenciales, siendo el estándar más seguro para clientes móviles.
- **Autorización / Encriptación:** Cifrado en tránsito con **HTTPS / TLS 1.3** desde el API Gateway. Cifrado en reposo para datos sensibles y biometría gestionado con **AWS KMS (Key Management)**.

9.4 Rendimiento

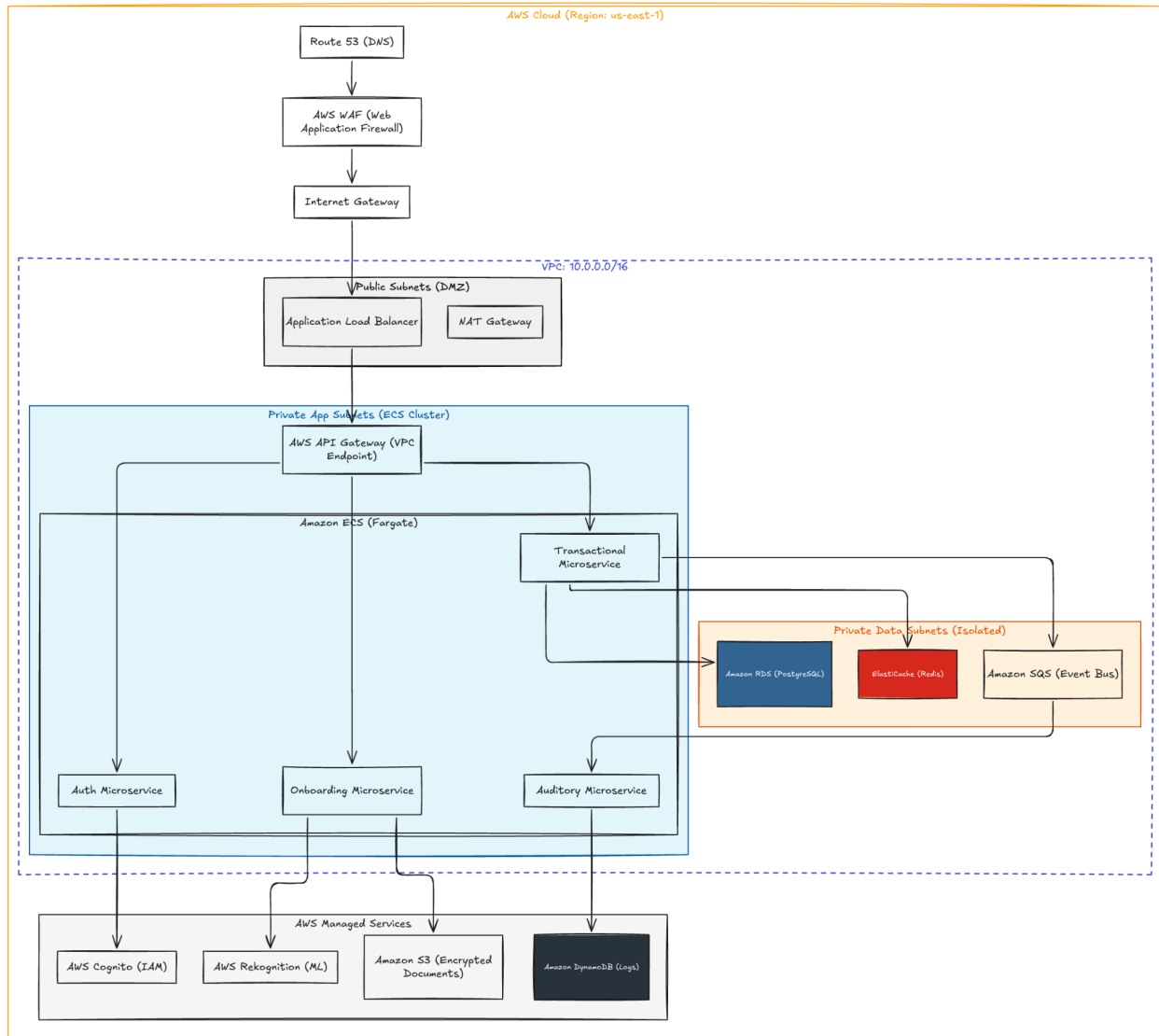
Se usa el patrón **Cache-Aside** soportado en **Redis (ElastiCache)** para persistir perfiles y saldos de "clientes frecuentes", entregando latencias de respuesta casi instantáneas y disminuyendo la carga hacia la plataforma Legacy.

10. Infraestructura y Despliegue

10.1 Arquitectura en la Nube

- **Proveedor Cloud:** AWS (Amazon Web Services).
- **Servicios Utilizados:** AWS Cognito (Autenticación), AWS API Gateway, Amazon ECS (Contenedores), Amazon RDS (Saldos relacionales), DynamoDB (Auditoría inmutable), ElastiCache/Redis (Caché), AWS Rekognition (Biometría), SQS/RabbitMQ (Mensajería asíncrona).

10.2 Diagrama de Despliegue (C4 - Nivel 4)



11. Decisiones de Arquitectura (ADR)

Decisión	Justificación Teórica (<i>Mínimo 2</i>)
Microservicios (Back-End)	1. Permiten crecimiento independiente de los componentes sin afectar al resto. 2. Asegura una estricta segmentación de responsabilidades de negocio.

Decisión	Justificación Teórica (<i>Mínimo 2</i>)
Flutter (App Móvil)	1. Uso de un <i>single codebase</i> que reduce tiempos de desarrollo y costos de mantenimiento en iOS y Android. 2. Ofrece alto rendimiento de renderizado y una interfaz 100% consistente de forma nativa.
Angular (Web SPA)	1. Framework altamente estructurado que ofrece seguridad robusta <i>out-of-the-box</i> idónea para banca. 2. Su tipado estricto facilita el mantenimiento y escalabilidad a largo plazo.
Patrón Saga	1. Asegura la consistencia atómica y de datos distribuidos. 2. Garantiza la reversión automática de saldos si un eslabón falla, protegiendo al cliente.
OAuth 2.0	1. Es el estándar recomendado para clientes públicos (SPA/Móvil). 2. Previene la interceptación del código de autorización ya que no se puede almacenar un secreto en el frontend.

12. Riesgos y Consideraciones

- **Riesgos técnicos:** La lentitud extrema del **Core Bancario Legacy** heredado. (*Mitigación: Patrón Circuit Breaker y Caché*).
- **Dependencias:** Fallas o cambios en la API del servicio de terceros **AWS Rekognition** podrían entorpecer el proceso de Onboarding.
- **Limitaciones:** La consistencia eventual derivada del patrón Saga y de herramientas asíncronas de mensajería (SQS) requerirá un buen manejo de los estados de las transacciones (UI) para no confundir al usuario durante el procesamiento